Universidad Nacional de San Martín

Escuela de Ciencia y Tecnología (ECyT)

Licenciatura en Desarrollo de Software · Verificación y Validación de Software

Trabajo Práctico Integrador

ATALAYA 3.1.0

Estrategia de testing e informe de QA

Grupo E · Turno noche

Roman Cardozo

Valentino Geronzi

Gonzalo Ventrici

Nicolas Beda

Docentes: Ing. Maximiliano Di Ludovico · Ing. Alejandro Millán

Fecha de entrega: 13/11/2026

Contenido

1. Introducción
2. Sistema bajo prueba y alcance
3. Fundamentos aplicados
4. Análisis de riesgos
5. Revisión de requerimientos
6. Plan de pruebas
7. Diseño de pruebas
8. Casos de prueba
9. Criterios de aceptación y BDD
10. Automatización, TDD y API
11. Defectos, Definition of Done y métricas
12. Informe final
13. Uso de inteligencia artificial
14. Entrega

1. Introducción

ATALAYA es un sistema de asistencia a la operación de una central nuclear: recibe señales de planta, estima la probabilidad de distintas desviaciones y genera alertas para la sala de control. No tiene ningún camino de escritura hacia los sistemas de control ni de protección, así que el riesgo que nos importa es que el operador tome una mala decisión por una alerta falsa, una alerta que no llega o que llega tarde, o por exceso de alertas.

El enunciado no trae código y los lineamientos piden entregar el SISTEMA en Java 21 con Spring Boot. Por eso implementamos una versión reducida de ATALAYA que cubre la lógica de mayor riesgo y, sobre ella, planificamos, diseñamos y ejecutamos las pruebas.

2. Sistema bajo prueba y alcance

El SISTEMA es una API REST con Spring Boot, persistencia en H2 y autenticación por roles con Spring Security. Las estimaciones del modelo de IA no se calculan: en el perfil qa se inyectan por un endpoint, igual que el avance del reloj, que es un java.time.Clock inyectado en las clases que manejan tiempos. Así podemos probar 45 s o 3 min sin esperar.

Incluido en el alcance	Fuera del alcance y motivo
RF-01/02: validación de rangos, duplicados y orden de llegada	RF-04/05 modo operativo y calibración: menor riesgo para la lógica de alertas
RF-03: redundancia de tres canales	RF-10 modo degradado y RF-11 integraciones y auditoría: dependen de sistemas externos
RF-06: niveles de riesgo y factores	Modelo de IA real y su evaluación estadística: no disponible
RF-07/08: alertas, repetición y escalamiento	Panel web: sin interfaz no aplican pruebas con Selenium
RF-09: agrupación en maniobras (nuevo en 3.1.0)	Desempeño, disponibilidad 99,95 % y carga: requieren infraestructura real
RF-12: umbrales con doble aprobación	Red segregada, licenciamiento y seguridad nuclear
Seguridad: autenticación y permisos por rol	

Endpoint	Uso
POST /api/v1/lecturas	Ingreso de una lectura (canal, magnitud, valor, secuencia)
GET /api/v1/magnitudes/{magnitud}	Valor consolidado y estado de cada canal
GET /api/v1/alertas · POST /api/v1/alertas/{id}/reconocimiento	Consultar y reconocer alertas
POST /api/v1/maniobras · DELETE /api/v1/maniobras/{id}	Declarar y finalizar maniobras
PUT /api/v1/config/umbrales · POST …/{id}/aprobacion	Solicitar y aprobar cambios de umbral
POST /api/v1/qa/estimaciones · POST /api/v1/qa/reloj	Solo perfil qa: inyectar estimaciones y avanzar el reloj

3. Fundamentos aplicados

Verificamos con revisión de requerimientos, revisión de código y pruebas contra cada RF. Validamos con criterios de aceptación escritos para que operación y seguridad puedan leerlos (sección 8). En el informe usamos error para la equivocación de una persona, defecto para lo que queda mal en un

artefacto y falla para lo que se observa al ejecutar. Por ejemplo: interpretar “más del 2 %” como “2 % o más” es un error; escribir >= en vez de > en el validador es el defecto; que un canal al 2,00 % quede como no confiable es la falla (CP-08).

Así aplicamos los principios del testing en este trabajo:

Principio	Cómo lo aplicamos
Las pruebas muestran la presencia de defectos, no su ausencia	En el informe final decimos qué no pudimos verificar
No se puede probar todo	Valores límite, partición de equivalencia y pairwise
Probar temprano	Revisamos los requerimientos antes de codificar (sección 5)
Los defectos se agrupan	Más casos en redundancia y escalamiento, donde están los bordes
Paradoja del pesticida	Revisamos los mutantes que sobreviven y agregamos casos
Las pruebas dependen del contexto	La severidad se mide por el efecto sobre la decisión del operador

Las características de calidad de ISO/IEC 25010 que priorizamos para este SISTEMA son las siguientes:

Característica	Subcaracterísticas	Prioridad	Cómo la evaluamos
Adecuación funcional	Corrección, completitud	Alta	Casos de caja negra y blanca sobre los RF del alcance
Fiabilidad	Tolerancia a fallos, madurez	Alta	Canales sin actualizar, duplicados, lecturas fuera de orden
Seguridad	Autenticidad, confidencialidad, responsabilidad	Alta	Permisos por rol, registro de quién reconoce, endpoints de QA apagados en producción
Mantenibilidad	Capacidad de ser probado, modificabilidad	Media	Reloj inyectable, cobertura y mutación
Usabilidad	Operabilidad	Baja en este alcance	Sin interfaz web; queda como mejora
Eficiencia de desempeño	Comportamiento temporal	Baja en este alcance	Requiere infraestructura real; queda fuera

Para el análisis estático usamos [herramienta vista en clase] antes de cada merge, y todo cambio pasa por un pull request revisado por otro integrante con esta checklist:

Punto de revisión	Por qué
Los bordes usan el operador que pide el requerimiento (> o >=)	Es el defecto más probable en RF-03, RF-06 y RF-08
Las clases con tiempos reciben el Clock y no llaman a now() directo	Si no, las pruebas de tiempo no son repetibles
Se validan nulos y valores fuera de rango en la entrada	RF-01/02
Cada prueba tiene comentario con su propósito y el ID del caso	Lo piden los lineamientos y ayuda a la trazabilidad
Los endpoints de QA están anotados con el perfil qa	Evita que lleguen a producción (CP-36)

4. Análisis de riesgos

Priorizamos por severidad (efecto sobre la decisión del operador) y probabilidad.

ID	Riesgo	Sev.	Prob.	Cómo lo cubrimos
R1	Alerta crítica que no llega o llega tarde	Alta	Media	Bordes de nivel y de escalamiento (CP-13, CP-21)
R2	Exceso de alertas repetidas: el operador deja de prestarles atención	Alta	Alta	Repetición de 3 min y agrupación en maniobras (CP-19, CP-27)
R3	La agrupación de maniobras oculta un desvío real	Alta	Media	Bordes de la banda, maniobra con falla de instrumento, pairwise
R4	Se usa un canal no confiable para estimar	Alta	Media	Tabla de decisión de RF-03
R5	Lecturas duplicadas, desordenadas o fuera de rango	Alta	Alta	CP-01 a CP-05
R6	Escalamiento a destiempo o duplicado	Alta	Media	Reloj simulado y caja blanca (CP-21 a CP-26)
R7	Cambio de umbral sin la aprobación requerida	Alta	Baja	CP-32 y CP-33
R8	Acceso con un rol indebido	Media	Baja	CP-34 a CP-36
R9	Datos sintéticos poco representativos	Media	Alta	Se declara como limitación

5. Revisión de requerimientos

La revisión de requerimientos fue la primera prueba que hicimos. Cada ambigüedad se cargó en Jira como defecto de especificación. Hasta tener respuesta, el SISTEMA implementa la decisión de la tabla.

#	Ambigüedad	Decisión
A1	“Discrepa más del 2 % del rango respecto de los otros”: ¿contra qué canal o valor?	Contra la mediana de los tres
A2	“2 % del rango”: ¿de qué rango?	Rango físico de RF-01 (330 °C → 6,6 °C)
A3	Al recuperarse un canal, ¿cuántas lecturas buenas hacen falta?	3 lecturas seguidas
A4	“Salvo que empeore”: ¿cambio de nivel o aumento de probabilidad?	Cambio a un nivel mayor
A5	Dos reconocimientos simultáneos de la misma alerta	Vale el primero; el segundo recibe 409
A6	Maniobra declarada que nunca se ejecuta	Expira a los 30 min
A7	Con alto en 30 %, ¿atención pasa a ser de 15 % a 30 %?	Sí
A8	El umbral crítico no tiene máximo	Máximo 100 %

6. Plan de pruebas

El objetivo es reunir evidencia suficiente sobre la lógica de mayor riesgo para poder recomendar si se libera o no. Probamos lo incluido en la sección 2. La estrategia es basada en riesgos: primero redundancia, niveles y escalamiento, después maniobras y configuración.

No probamos lo que quedó fuera del alcance en la sección 2, ni el modelo de IA. Sobre el modelo, el requerimiento pide una sensibilidad mínima del 95 %, y con pocos eventos críticos ese número es engañoso: 19 detecciones sobre 20 dan 95 %, pero el intervalo de confianza va aproximadamente de 75 % a 99,9 %. Para afirmar que la sensibilidad es de al menos 95 % con 95 % de confianza harían falta unos 59 eventos detectados sin ninguna falla. Lo dejamos planteado como método para cuando el modelo esté disponible.

Para empezar a ejecutar pedimos que el proyecto compile con mvn verify, que exista el perfil qa y que los casos estén revisados por otro integrante. Para terminar, que todos los casos estén ejecutados, que no haya defectos críticos abiertos y que la cobertura y la mutación superen lo definido en la sección 11. Suspendemos si el build falla o si un defecto bloquea el resto de las pruebas, y retomamos repitiendo los casos afectados.

Entorno	Uso
Desarrollo	Cada integrante en VS Code con Java 21; pruebas unitarias y TDD
QA	Perfil qa con reloj y estimaciones inyectables; suite completa, Postman
Staging	Perfil prod; se verifica que los endpoints de QA no existan (CP-36)
Producción	Sala de control; fuera del alcance del trabajo

Integrante	Rol
Roman Cardozo	[rol]
Valentino Geronzi	[rol]
Gonzalo Ventrici	[rol]
Nicolas Beda	[rol]

Estimamos el esfuerzo en horas de equipo:

Fecha	Actividad	Esfuerzo
30/10	Revisión con los docentes y cierre de alcance	4 h
31/10 al 05/11	Proyecto Maven, RF-01 a RF-03 y RF-06 con TDD; casos manuales en Postman	24 h
06/11 al 09/11	RF-07/08, RF-09 y RF-12; Cucumber; defectos en Jira	24 h
10/11 al 11/11	Ejecución completa, JaCoCo y PIT, análisis de mutantes	12 h
12/11	Informe final y ensayo de la exposición	8 h
13/11	Entrega en el campus	1 h

Herramientas que usamos:

Herramienta	Uso
Java 21, Spring Boot, Maven	SISTEMA y ejecución de todas las pruebas

Herramienta	Uso
JUnit 5, Spring Boot Test, MockMvc	Pruebas unitarias, de integración y de API
Cucumber	Criterios de aceptación en Dado-Cuando-Entonces
JaCoCo y PIT	Cobertura y mutación
Postman	Ejecución manual y automatizada de la API
Jira	Tareas, defectos y tablero
[herramienta de pairwise]	Generación de combinaciones

Los cambios de alcance o de requerimientos se cargan en Jira, se revisa qué casos afectan en la matriz de trazabilidad y se vuelve a correr la regresión. Para el release trabajamos con ramas por funcionalidad, merge por pull request con el build en verde y etiquetas 3.1.0-rc1, rc2 hasta cumplir los criterios de salida.

7. Diseño de pruebas

Probamos en cuatro niveles. Unitario: las clases de dominio con JUnit 5. Integración: los servicios con H2, usando Spring Boot Test. Sistema: la API con MockMvc y Postman. Aceptación: escenarios en Cucumber. Las técnicas de caja negra se aplican donde hay reglas con bordes claros, la caja gris donde importa lo que queda guardado en la base, y la caja blanca en las dos clases con más decisiones. No hacemos combinatoria completa (serían 162 casos) ni esperas en tiempo real, que son lentas y frágiles.

7.0 Casos de uso

Los casos de prueba salen de estos casos de uso. En cada uno indicamos el flujo principal y los alternativos que probamos.

CU	Actor	Flujo principal	Flujos alternativos
CU-01 Ingresar lecturas (RF-01 a RF-03)	Sistema de adquisición	Envía una lectura válida de un canal; el SISTEMA la guarda y recalcula el estado de los canales	Valor fuera de rango, mensaje inválido, duplicado, fuera de orden, canal que discrepa o deja de actualizarse
CU-02 Consultar nivel de riesgo (RF-06)	Operador de reactor	Consulta la estimación vigente y ve el nivel y, desde atención, los tres factores principales	Probabilidad inválida; umbrales ajustados
CU-03 Reconocer y escalar alertas (RF-07/08)	Operador, jefe de turno, supervisor de SO	Recibe la alerta y la reconoce dentro del plazo	No la reconoce y escala; la alerta empeora; se repite; dos reconocimientos a la vez
CU-04 Declarar una maniobra (RF-09)	Jefe de turno	Declara la maniobra con su banda; las variaciones dentro de la banda se agrupan	Desvío fuera de banda; falla de un canal; la maniobra no se ejecuta y expira
CU-05 Cambiar umbrales (RF-12)	Supervisor de SO y jefe de turno	El supervisor solicita, el jefe aprueba y el umbral cambia	Valor fuera de límites; falta la segunda aprobación; aprueba el mismo usuario
CU-06 Acceder al SISTEMA	Todos los roles	Se autentica y opera según su rol	Sin credenciales; rol sin permiso; endpoints de QA en producción

Trazabilidad entre requerimientos y casos (la matriz completa, con resultados y defectos, está en la planilla):

Requerimiento	Casos
RF-01/02	CP-01 a CP-05
RF-03	CP-06 a CP-12, CP-29, CP-31
RF-06	CP-13 a CP-17
RF-07/08	CP-18 a CP-26, CP-31
RF-09	CP-27 a CP-31
RF-12	CP-16, CP-32, CP-33
RNF seguridad	CP-34 a CP-36

7.1 Tabla de decisión del estado de un canal (RF-03)

Condición	R1	R2	R3	R4
Actualizado en los últimos 10 s	S	N	S	S

Condición	R1	R2	R3	R4
Dentro del rango físico	S	-	N	S
Discrepancia con la mediana ≤ 2 %	S	-	-	N
Estado	Confiable	No confiable	No confiable	No confiable
Caso	CP-06	CP-09	CP-10	CP-07

7.2 Estados de una alerta alta o crítica

Estado	Evento	Siguiente	Caso
Emitida	Reconocimiento	Reconocida	CP-23
Emitida	Vence el plazo (45 s o 90 s)	Escalada	CP-21, CP-22
Escalada	Reconocimiento	Reconocida	CP-31
Emitida o escalada	Sube de nivel	Nueva alerta del nivel mayor	CP-20
Reconocida	Reconocimiento	Sin cambio (409)	CP-24

7.3 Caja blanca

Para ClasificadorNivel tomamos el código y calculamos la complejidad ciclomática: cuatro decisiones, V(G) = 5. Elegimos un dato por cada camino básico y agregamos p = 100,1 para cubrir la segunda condición de D1 (CP-17). Con ServicioEscalamiento hicimos lo mismo apuntando a cobertura de decisiones (CP-26).

```
public NivelRiesgo clasificar(double p, Umbrales u) {
    if (p < 0 || p > 100) throw new IllegalArgumentException();   // D1
    if (p >= u.critico()) return NivelRiesgo.CRITICO;               // D2
    if (p >= u.alto())    return NivelRiesgo.ALTO;                  // D3
    if (p >= 15.0)        return NivelRiesgo.ATENCION;              // D4
    return NivelRiesgo.NORMAL;
}
```

Camino	Recorrido	Dato	Resultado
C1	D1 verdadera	−1	Excepción
C2	D2 verdadera	80	CRITICO
C3	D3 verdadera	50	ALTO
C4	D4 verdadera	20	ATENCION
C5	Todas falsas	5	NORMAL

7.4 Caja gris

Verificamos en la base H2 que las lecturas duplicadas no se guarden dos veces (CP-04), que el reconocimiento quede con usuario y hora (CP-23) y que dos reconocimientos simultáneos no dejen dos registros (CP-25). También revisamos la estructura de las respuestas JSON y que los endpoints de QA no existan con el perfil de producción (CP-36).

7.5 Pairwise

En la generación y el escalamiento de alertas se cruzan cinco parámetros: maniobra (3 valores), canales (3), rol que reconoce (3), momento del reconocimiento (3) y nivel (2). Con [herramienta: PICT o allpairspy] bajamos de 162 combinaciones a 10, y verificamos que cubren todos los pares (CP-31).

Fila	Maniobra	Canales	Reconoce	Cuándo	Nivel	Esperado
1	Ninguna	3 confiables	Operador	Antes	Alto	Reconocida sin escalar
2	Ninguna	1 no confiable	Jefe	Después	Alto	Aviso; escala a 90 s; reconocida
3	Ninguna	Menos de 2	Superv. SO	Nunca	Alto	Aviso; escala a 90 s; queda escalada
4	Expirada	3 confiables	Jefe	Nunca	Alto	Escala a 90 s; queda escalada
5	Expirada	1 no confiable	Superv. SO	Antes	Alto	Aviso; reconocida sin escalar
6	Expirada	Menos de 2	Operador	Después	Crítico	Aviso; escala a 45 s; reconocida
7	Activa	3 confiables	Superv. SO	Después	Alto	No se agrupa; escala a 90 s; reconocida
8	Activa	1 no confiable	Operador	Nunca	Crítico	Aviso aparte; escala a 45 s; queda escalada
9	Activa	Menos de 2	Jefe	Antes	Crítico	Aviso; no se agrupa; reconocida sin escalar
10	Ninguna	3 confiables	Superv. SO	Antes	Crítico	Reconocida sin escalar

7.6 Permisos por rol

Operación	Adquisición	Operador	Jefe de turno	Superv. SO	Auditor
Enviar lecturas	Sí	No	No	No	No
Consultar magnitudes y alertas	No	Sí	Sí	Sí	Sí
Reconocer alertas	No	Sí	Sí	Sí	No
Declarar maniobra	No	No	Sí	No	No
Solicitar cambio de umbral	No	No	No	Sí	No
Aprobar cambio de umbral	No	No	Sí	No	No

8. Casos de prueba

Diseñamos 36 casos. En la planilla Matriz_Trazabilidad_ATALAYA.xlsx cada uno tiene identificador, requerimiento, precondiciones, datos de prueba, pasos, resultado esperado y criterio de aprobación, más las columnas de resultado, evidencia y defecto que completamos al ejecutar. La precondición "Base" es: SISTEMA levantado con perfil qa, base H2 limpia, reloj de prueba en T0, sin maniobra activa, umbrales por defecto (alto 40 %, crítico 75 %) y usuario autenticado con el rol indicado. Los datos son todos sintéticos.

ID	Req.	Caso	Técnica	Caja	Ejecución
CP-01	RF-01/02	Límites de temperatura del primario	Valores límite	Negra	Postman + JUnit
CP-02	RF-01/02	Límites de presión, presurizador, potencia y dosis	Valores límite	Negra	JUnit (parametrizado)
CP-03	RF-01/02	Mensajes inválidos	Partición de equivalencia	Negra	Postman + MockMvc
CP-04	RF-01/02	Lectura duplicada	Adivinanza de errores	Gris	JUnit + consulta a H2
CP-05	RF-01/02	Lectura fuera de orden	Adivinanza de errores	Negra	JUnit
CP-06	RF-03	Tres canales coherentes	Tabla de decisión (R1)	Negra	JUnit
CP-07	RF-03	Canal que discrepa 2,01 %	Tabla de decisión (R4) + valores límite	Negra	Postman + JUnit
CP-08	RF-03	Discrepancia de exactamente 2,00 %	Valores límite	Negra	JUnit
CP-09	RF-03	Canal sin actualizar 10 s y 10,1 s	Tabla de decisión (R2) + límites de tiempo	Negra	JUnit (reloj simulado)
CP-10	RF-03	Valor fuera de rango en un canal	Tabla de decisión (R3)	Negra	JUnit
CP-11	RF-03	Menos de dos canales confiables	Tabla de decisión	Negra	Postman + JUnit
CP-12	RF-03	Recuperación de un canal	Transición de estados	Negra	JUnit
CP-13	RF-06	Bordes de nivel	Valores límite	Negra	Postman + JUnit
CP-14	RF-06	Probabilidad inválida	Partición de equivalencia	Negra	JUnit
CP-15	RF-06	Tres factores desde atención	Partición de equivalencia	Negra	JUnit
CP-16	RF-06, RF-12	Niveles con umbrales ajustados	Valores límite	Negra	JUnit
CP-17	RF-06	Caminos básicos de ClasificadorNivel	Caja blanca: caminos (V(G)=5)	Blanca	JUnit + JaCoCo
CP-18	RF-07	Destinatarios según nivel	Partición de equivalencia	Negra	Postman + JUnit
CP-19	RF-07	No repetir durante 3 min	Valores límite de tiempo	Negra	JUnit (reloj simulado)
CP-20	RF-07	Empeora antes de los 3 min	Transición de estados	Negra	JUnit
CP-21	RF-08	Escalamiento de crítico a 45 s	Valores límite de tiempo	Negra	JUnit + Cucumber
CP-22	RF-08	Escalamiento de alto a 90 s	Valores límite de tiempo	Negra	JUnit (reloj simulado)
CP-23	RF-08	Reconocimiento antes del plazo	Transición de estados	Gris	Postman + JUnit
CP-24	RF-08	Reconocer una alerta ya reconocida	Transición de estados	Negra	MockMvc
CP-25	RF-08	Dos reconocimientos simultáneos	Adivinanza de errores	Gris	JUnit (2 hilos) + consulta a H2
CP-26	RF-08	Decisiones de ServicioEscalamiento	Caja blanca: decisiones	Blanca	JUnit + JaCoCo + PIT
CP-27	RF-09	Variaciones dentro de la banda	Partición de equivalencia	Negra	Postman + Cucumber
CP-28	RF-09	Bordes de la banda	Valores límite	Negra	JUnit
CP-29	RF-09, RF-03	Maniobra con canal no confiable	Combinatoria	Negra	JUnit
CP-30	RF-09	Maniobra que nunca se ejecuta	Valores límite de tiempo	Negra	JUnit (reloj simulado)
CP-31	RF-03, RF-07/08, RF-09	Combinaciones pairwise	Pairwise	Negra	JUnit (CSV) + Postman

ID	Req.	Caso	Técnica	Caja	Ejecución
CP-32	RF-12	Límites de umbrales	Valores límite	Negra	Postman + JUnit
CP-33	RF-12	Doble aprobación	Tabla de decisión	Negra	JUnit + Cucumber
CP-34	RNF seguridad	Acceso sin autenticar	Partición de equivalencia	Negra	Postman + MockMvc
CP-35	RNF seguridad	Permisos por rol	Tabla de decisión	Negra	JUnit (parametrizado)
CP-36	RNF seguridad	Endpoints de QA apagados en producción	Partición de equivalencia	Gris	JUnit (perfil prod)

Tres casos completos como ejemplo:

Campo	CP-07 · Canal que discrepa 2,01 %
Requerimiento	RF-03
Técnica y caja	Tabla de decisión (R4) + valores límite · caja negra
Ejecución	Postman + JUnit
Precondiciones	Base
Datos	A=B=300,00; C=306,64 °C
Pasos	Enviar y consultar
Resultado esperado	C NO_CONFIABLE por DISCREPANCIA; el valor usa A y B
Criterio de aprobación	Estado de C y valor consolidado

Campo	CP-21 · Escalamiento de crítico a 45 s
Requerimiento	RF-08
Técnica y caja	Valores límite de tiempo · caja negra
Ejecución	JUnit + Cucumber
Precondiciones	Alerta CRITICO sin reconocer en T0
Datos	Reloj en 44, 45 y 46 s
Pasos	Avanzar el reloj y consultar destinatarios
Resultado esperado	44 s sin escalar; 45 s escala al supervisor de seguridad operacional; 46 s no se duplica
Criterio de aprobación	Un único escalamiento a los 45 s

Campo	CP-33 · Doble aprobación
Requerimiento	RF-12
Técnica y caja	Tabla de decisión · caja negra
Ejecución	JUnit + Cucumber
Precondiciones	Solicitud de alto 35 % creada por el supervisor de SO
Datos	Sin aprobar; aprueba el mismo usuario; aprueba el jefe de turno
Pasos	Aplicar cada variante y consultar umbrales
Resultado esperado	Solo con la aprobación del jefe de turno pasa a 35 %
Criterio de aprobación	Umbral vigente en cada variante

Cuando la salida es una estimación no esperamos un número exacto sino un nivel. Por eso los casos de RF-06 trabajan con la probabilidad inyectada y verifican el nivel y los factores, no el cálculo del modelo.

9. Criterios de aceptación y BDD

Criterio de aceptación	Caso
Un canal que discrepa más del 2 % se deja de usar y se avisa	CP-07
Una alerta crítica sin reconocer escala al supervisor de seguridad operacional a los 45 s	CP-21
Durante una maniobra, las variaciones esperables generan una sola alerta informativa	CP-27
Un umbral solo cambia con la aprobación de un segundo rol	CP-33

```
# language: es
Característica: Escalamiento de alertas críticas
  Esquema del escenario: Bordes del escalamiento
    Dado una alerta crítica emitida al operador de reactor
    Cuando pasan <segundos> segundos sin reconocimiento
    Entonces el estado de la alerta es <estado>
    Ejemplos:
      | segundos | estado   |
      | 44       | EMITIDA  |
      | 45       | ESCALADA |
```

10. Automatización, TDD y API

Automatizamos todo lo que tiene un resultado verificable sin juicio humano: reglas de dominio, servicios y API. Usamos JUnit 5 con pruebas parametrizadas, Spring Boot Test y MockMvc, Cucumber para los criterios de aceptación, JaCoCo para la cobertura y PIT para la mutación. Las pruebas de caja negra marcadas con Postman se ejecutan también a mano, guardando la captura de cada respuesta como evidencia. Todo corre con mvn verify y cada clase de prueba tiene un comentario con su propósito y el ID del caso.

Aplicamos TDD en ValidadorRedundancia y ServicioEscalamiento, que son reglas precisas con bordes numéricos y de tiempo. Dejamos un commit por cada etapa (red, green, refactor) con la captura de la ejecución que falla y de la que pasa.

La colección de Postman tiene una carpeta por requerimiento, un entorno con baseUrl y usuarios por rol, y scripts que verifican código HTTP y cuerpo de la respuesta. Se exportan la colección y el entorno.

10.1 Ejecución manual y evidencia

Los casos de caja negra que figuran con Postman se ejecutan a mano antes de automatizarlos. Para cada ejecución anotamos en la planilla la fecha, quién lo ejecutó, el resultado (aprobado, fallido o bloqueado), la ruta de la captura y, si falló, la clave del defecto en Jira. Para que sea repetible, la precondición se prepara siempre con los endpoints del perfil qa y con el reloj en T0.

10.2 Mutation testing con PIT

PIT se corre sobre el paquete de dominio. Nos interesan sobre todo los mutantes que cambian bordes, porque son los defectos más probables en este SISTEMA:

Mutante	Dónde	Caso que debería matarlo
>= pasa a > en 15, 40 o 75 %	ClasificadorNivel	CP-13

Mutante	Dónde	Caso que debería matarlo
> pasa a >= en el 2 %	ValidadorRedundancia	CP-08
45 s pasa a 46 s	ServicioEscalamiento	CP-21
Se elimina la condición “ya reconocida”	ServicioEscalamiento	CP-24
30 % pasa a 29 % en el límite del umbral alto	ValidadorUmbrales	CP-32

Resultado: [mutation score, mutantes sobrevivientes y casos agregados].

11. Defectos, Definition of Done y métricas

Los defectos se registran en Jira con versión, caso, pasos, resultado esperado y observado, evidencia, severidad y prioridad. La severidad es crítica si puede llevar al operador a una mala decisión, alta si demora o duplica una alerta, media si es un error sin efecto directo y baja si es cosmético. El flujo es Nuevo, Confirmado, En corrección, Corregido, Verificado y Cerrado. Si una prueba sigue fallando al entregar, es porque evidencia un defecto reportado, y lo indicamos en su nombre.

Defecto de especificación	Severidad
RF-03 no define contra qué ni sobre qué rango se mide el 2 % (A1, A2)	Alta
RF-07 no define qué significa que una alerta “empeore” (A4)	Alta

Ejemplo de reporte (defecto de especificación de la revisión):

Campo	Contenido
Clave y resumen	[ATA-1] RF-03 no define contra qué ni sobre qué rango se mide el 2 %
Tipo	Defecto de especificación
Requerimiento y caso	RF-03 · CP-07, CP-08
Descripción	El requerimiento dice “discrepa más del 2 % del rango respecto de los otros”. Con tres canales no queda claro si se compara contra la mediana, el promedio o cada par, ni si el rango es el físico o el del instrumento.
Impacto	Dos implementaciones correctas según el texto marcan canales distintos como no confiables, y eso cambia la estimación.
Severidad y prioridad	Alta · Alta
Estado	Confirmado; se pidió aclaración a la cátedra. Mientras tanto se implementa la decisión A1/A2.

Consideramos terminado un caso cuando está diseñado y trazado, lo revisó otro integrante, se ejecutó con evidencia, sus defectos están en Jira y el build está en verde.

Indicador	Umbral
Casos ejecutados (aprobados, fallidos, bloqueados)	100 % ejecutados, 0 bloqueados
Defectos abiertos por severidad	0 críticos
Cobertura de sentencias y ramas (JaCoCo) en dominio	≥ 80 % y ≥ 70 %
Mutation score (PIT) en dominio	≥ 75 %
Requerimientos del alcance con casos ejecutados	100 %

La cobertura dice qué código se ejecutó, no si las verificaciones son buenas; por eso la miramos junto con la mutación. Los mutantes que sobreviven se analizan y, si muestran un caso faltante, lo agregamos.

12. Informe final

Resultado	Valor
Casos aprobados / fallidos / bloqueados	[completar]
Defectos por severidad	[completar]
Cobertura JaCoCo (sentencias / ramas)	[completar]
Mutation score PIT	[completar]

Riesgos residuales:

- El comportamiento del modelo real, que no pudimos probar.
- Los requerimientos que quedaron fuera del alcance (RF-04/05, RF-10, RF-11).
- Las ambigüedades de la sección 5 que la cátedra no llegue a aclarar.

Limitaciones:

- El SISTEMA es una implementación reducida hecha por nosotros, no el ATALAYA real.
- Los datos son sintéticos.
- El mismo equipo desarrolla y prueba, lo que resta independencia.

Recomendación de liberación: [completar con los resultados].

Como mejora, queda probar los requerimientos excluidos, sumar una interfaz web con pruebas de Selenium y evaluar el modelo real con datos de transitorios y arranques.

13. Uso de inteligencia artificial

Usamos IA como apoyo, según la condición 11.f de los lineamientos.

Herramienta	Para qué	Qué verificamos
[herramienta]	[uso]	[verificación del equipo]

14. Entrega

Se entrega VV-Q2-2026-TN-GrupoE-ATALAYA.zip con el proyecto Maven (código del SISTEMA y de las pruebas), la colección y el entorno de Postman, los reportes de JaCoCo, PIT y Cucumber, la planilla de casos y trazabilidad, las evidencias y este informe.

```
VV-Q2-2026-TN-GrupoE-ATALAYA/
  pom.xml
  src/main/java/...            SISTEMA
  src/test/java/...            pruebas JUnit, MockMvc y Cucumber
  src/test/resources/features  escenarios .feature
  postman/                     colección y entorno exportados
  reportes/                    jacoco, pit-reports, cucumber
  docs/                        informe, planilla de casos, evidencias
```